

Universidad Centroccidental "Lisandro Alvarado" (UCLA)

Asignatura: Laboratorio I

Profesor: Jonathan Falcon

Lapso Académico: 2026-1

Enunciado de Proyecto Laboratorio I 2026-1 — Plataforma API para Gestión Integral de Gimnasios (SmartGym)

1. Descripción general

El objetivo de este proyecto es diseñar y construir una API robusta y escalable que permita la gestión operativa, financiera y administrativa de un gimnasio moderno. La solución deberá modelar un dominio que involucra control de acceso físico, gestión de inventarios, membresías, un sistema de reservas de clases con restricciones de concurrencia y seguimiento biométrico.

La API deberá estar documentada bajo el estándar OpenAPI/Swagger y protegida mediante autenticación robusta por tokens.

Consideraciones Técnicas Obligatorias

- **Lenguajes y Frameworks Backend permitidos:** El equipo deberá justificar la elección de su stack. Se recomiendan:
 - **Node.js** (Express, NestJS o Fastify).
 - **.NET / C#** (ASP.NET Core Web API).
 - **Python** (FastAPI, Django REST Framework o Flask).
 - **Go** (Gin o Fiber).
- **Motor de Base de Datos:** Debe ser estrictamente relacional. Las opciones permitidas son **PostgreSQL**, **Microsoft SQL Server** o **MySQL**. Se evaluará el uso correcto de llaves foráneas, índices y restricciones de integridad.
- **Control de versiones (Git): Regla estricta de repositorio:** El trabajo colaborativo no será con *fork*; los estudiantes miembros del equipo tendrán acceso directo de escritura al repositorio desde un inicio para gestionar sus ramas.
- **Nomenclatura del repositorio:** lab1-proyecto-2026-1-cedula1-cedula2-cedula3

2. Requerimientos funcionales

Los estudiantes deberán analizar los siguientes módulos y deducir los procesos, entidades y flujos necesarios para satisfacer la operativa del gimnasio.

2.1 Módulo de Identidad, Autenticación y Autorización

- El sistema debe soportar la creación de usuarios con acceso al sistema mediante credenciales.
- Debe implementarse un sistema de roles estandarizado. Los roles principales requeridos son: **Administración, Finanzas, Entrenadores y Clientes**.
- El acceso a los recursos de la API debe estar protegido mediante autenticación por *Bearer Token* (JWT).
- Se debe aplicar control de acceso basado en roles (RBAC). Por ejemplo, un Cliente no debe poder modificar el inventario de máquinas, y un Entrenador no debería tener acceso al módulo de finanzas.

2.2 Módulo de Inventario e Instalaciones

- Gestión completa del equipamiento del gimnasio.
- Las **Máquinas** deben estar catalogadas lógicamente. El sistema debe almacenar al menos el identificador, nombre de la máquina, descripción técnica, estado operativo (activa, en mantenimiento, fuera de servicio) y su categoría (ej. Cardiovascular, Musculación, Peso Libre).

2.3 Módulo de Gestión Deportiva (Clases y Entrenadores)

- Debe existir un catálogo de disciplinas o tipos de clases (ej. Spinning, Yoga, Crossfit).
- Los administradores o coordinadores deben poder programar **Sesiones de Clases** en horarios específicos.
- Cada sesión debe tener asignado a un **Entrenador** responsable y contar con un límite máximo de **Cupos** (ej. Clase de Spinning a las 7:00 PM con el Entrenador XXX, límite de 30 personas).

2.4 Módulo de Reservas y Clientes

- Los clientes deben poder consultar la disponibilidad e inscribirse en las sesiones de clases programadas.

- **Reglas de concurrencia:** El sistema debe impedir estrictamente que un mismo cliente reserve cupo en dos clases distintas que compartan el mismo bloque horario total o parcialmente. Sí pueden reservar múltiples clases en el mismo día, siempre que los horarios no se solapen.
- El sistema debe gestionar la lista de asistentes y no permitir inscripciones si la clase ha alcanzado su capacidad máxima.

2.5 Módulo de Control de Acceso

- El gimnasio requiere un registro de entrada. El sistema debe exponer un servicio que reciba el número de documento de identidad (Cédula) de la persona al pasar por el torniquete/recepción.
- Este servicio debe registrar automáticamente la fecha y hora exacta de entrada, y validar si la persona tiene permisos de acceso al recinto en ese momento (membresía activa).

2.6 Módulo de Finanzas y Planes de Suscripción

- El gimnasio debe poder ofertar distintos **Planes de Suscripción** (ej. Mensualidad Básica, Trimestre VIP, Pase Diario), definiendo sus costos y duración.
- Se debe llevar el registro de los pagos realizados por los clientes para adquirir o renovar dichos planes.
- El sistema debe ser capaz de determinar si la membresía de un cliente está "Activa", "Vencida" o "Por Vencer" basándose en su historial de pagos y la vigencia de su plan.

2.7 Módulo de Seguimiento Biométrico (Progresos)

- Los entrenadores deben poder registrar evaluaciones físicas periódicas de los clientes.
- El sistema debe almacenar métricas en el tiempo: peso, estatura, porcentaje de grasa corporal y observaciones generales.
- Debe permitir consultar el historial de evolución de un cliente específico, ordenado cronológicamente.

2.8 Módulo de Tienda (Punto de Venta - POS)

- El gimnasio vende productos físicos (botellas de agua, suplementos, toallas, franelas).

- Se debe gestionar un inventario de productos distinto al inventario de máquinas, controlando el stock disponible.
- El sistema debe permitir registrar una **Venta** a un cliente, descontando automáticamente el stock y registrando el ingreso en el sistema financiero.

2.9 Módulo de Mantenimiento de Instalaciones

- Las máquinas sufren desgaste. El sistema debe permitir abrir un "Ticket de Mantenimiento" para una máquina específica.
- Al abrir un ticket, el estado operativo de la máquina debe cambiar automáticamente a "En Mantenimiento" o "Fuera de Servicio", alertando a los administradores.
- Se debe guardar el historial de reparaciones por máquina (fecha de falla, descripción, fecha de resolución y costo de reparación).

3. Requerimientos no funcionales

- **Arquitectura:** Diseño en capas (Controladores, Servicios, Repositorios) o Arquitectura Limpia.
- **Seguridad:** Hasheo de contraseñas, sanitización de inputs para evitar inyecciones SQL, y validación de esquemas de datos entrantes.
- **Despliegue :** La solución debe estar contenerizada (Docker) y proporcionar un archivo docker-compose que levante tanto la API como el motor de base de datos de forma simultánea. (opcional) se puede levantar el proyecto de forma local

4. Entidades y Dominio Sugerido (MER)

El estudiante debe diseñar el Modelo Entidad-Relación considerando al menos las siguientes áreas conceptuales (el diseño exacto de tablas, llaves foráneas y tablas puente queda a evaluación):

1. **Seguridad:** Usuarios, Roles.
2. **Operativa Física:** Maquinas, CategoríasMaquinas, TicketsMantenimiento.
3. **Académico/Deportivo:** Disciplinas/TiposClase, Entrenadores, SesionesProgramadas (Clases), EvaluacionesBiometricas.
4. **Interacción del Cliente:** Clientes, Reservas/Inscripciones, ControlAccesos (Bitácora de entradas).
5. **Comercial:** PlanesSuscripcion, MembresiasCliente, Pagos/Facturas, ProductosTienda, VentasTienda.

5. Diseño de la API y Estándares de Comunicación

A diferencia de proyectos guiados, en este laboratorio **no se entregará la lista de rutas exactas**. Los equipos deberán aplicar las mejores prácticas RESTful para exponer el dominio. Se evaluará el uso correcto de URIs, jerarquías para recursos anidados, colecciones con filtrado y paginación.

Endpoints Sugeridos (Solo ejemplos ilustrativos)

- **Gestión de Clases:**
 - GET /api/v1/sesiones?fecha=2026-05-10&disciplina=spinning -> Lista sesiones aplicando filtros.
 - POST /api/v1/sesiones -> Crea una nueva clase en el calendario (Solo Admin).
- **Operaciones del Cliente (Reservas y Accesos):**
 - POST /api/v1/reservas -> Crea una inscripción ({ "clienteId": 105, "sesionId": 842 }).
 - POST /api/v1/accesos/entrada -> Registra el paso por recepción ({ "documentoIdentidad": "V-12345678" }).
- **Actualizaciones Parciales:**
 - PATCH /api/v1/maquinas/{id}/estado -> Modifica la disponibilidad ({ "estado": "EN_MANTENIMIENTO" }).

Estandarización de Códigos de Estado HTTP

- **200 OK:** Solicitud procesada correctamente.
- **201 Created:** Recurso creado exitosamente.
- **400 Bad Request:** Payload con errores de validación (ej. falta email).
- **401 Unauthorized:** Falta Token o token expirado.
- **403 Forbidden:** El usuario no tiene permisos para la acción.
- **404 Not Found:** El recurso no existe.
- **409 Conflict: Exclusivo para Reglas de Negocio fallidas.** (Ej. clase sin cupos, solapamiento de horarios, membresía vencida, producto sin stock).

Estructura de Error Estricta

Los errores HTTP 400 y 409 deben retornar un objeto JSON estructurado:

```
JSON
{
  "error": "Conflict",
  "codigoInterno": "ERR_RESERVA_CAPACIDAD_MAXIMA",
  "mensaje": "La clase de Crossfit de las 18:00 no tiene cupos disponibles.",
  "timestamp": "2026-05-10T14:32:00Z"
```

}

6. Reglas de Negocio Críticas (Evaluación de Lógica)

1. **Solapamiento:** Prohibido que un cliente se inscriba en dos clases que ocurran a la misma hora.
2. **Sobreventa:** No se pueden aceptar reservas en una sesión de clase si los cupos actuales son iguales al límite de la clase.
3. **Acceso Físico:** El endpoint de registro de entrada físico mediante cédula debe rechazar (retornar error lógico 409 Conflict) el acceso si el cliente no tiene una membresía pagada y vigente en ese momento.
4. **Auditoría Básica:** Todo pago registrado debe reflejar inmutablemente el monto, la fecha y el plan adquirido.

7. Documentación y Calidad

- **Swagger/OpenAPI:** Autogenerado e interactivo, accesible en una ruta específica (ej. /api-docs). Debe incluir los esquemas de *request* y *response* requeridos.
- **Seeders/Datos Semilla:** El proyecto debe incluir un mecanismo (script o migración) que al levantar la base de datos por primera vez cree automáticamente:
 - 1 Usuario Administrador.
 - Los Roles fundamentales.
 - 3 Categorías de máquinas y 5 máquinas de ejemplo.
 - 2 Planes de suscripción.
 - 3 Productos en la tienda.
- **README:** Instrucciones claras y paso a paso para levantar el proyecto desde cero utilizando Docker.

8. Entregables y Fechas

Primera Entrega — Viernes 24 de abril de 2026 / 20 pts

Contenido esperado (Análisis y Diseño):

1. Diagrama MER completo y normalizado.
2. Alcance del proyecto.
3. Requisitos funcionales y no funcionales

4. Diseño de la API: Listado de Endpoints propuestos (Ruta, Verbo, Descripción breve, Rol requerido).
5. Elección justificada del stack tecnológico.
6. Identificación clara de validaciones requeridas para cumplir con las reglas de negocio del gimnasio.

Nota: La primera entrega se hará por correo en un único documento PDF con el asunto "Primera Entrega Laboratorio I - [cedula1] - [cedula2]".

Segunda Entrega — martes 12 de mayo de 2026

Contenido esperado (Desarrollo Fase 1):

1. Implementación funcional del MER en base de datos.
2. Módulos de Seguridad (Auth/Roles) usuarios, Inventario de Máquinas, Planes de Suscripción operativos, Gestión de Clases, reserva de clases.

Entrega Final — martes 2 de junio de 2026

Proyecto completo, Dockerizado y documentado.

Se evaluarán de manera estricta las reglas lógicas complejas (solapamiento de clases, control de acceso por membresía, stock de tienda y registro de pagos).

IMPORTANTE: Se evaluará que los integrantes del proyecto tengan sus respectivos usuarios git y hagan su contribución en código, no está permitido subir con el usuario de otro compañero de grupo, ya que será tomado como falta en el proyecto